

Lab Sheet # L11

Instructions:

- Submit *.sql file, named as Lab11_6XXXXXX.sql to LAs or instructors prior to My Courses.
- If students cannot finish within the class time, students can submit until the beginning of the next class.

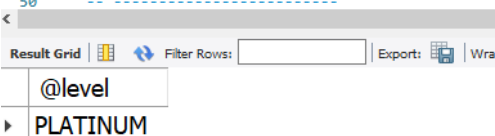
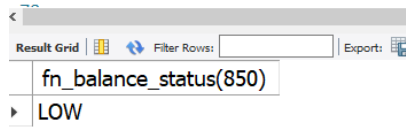
Self-study:

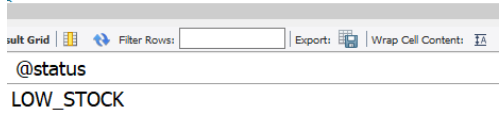
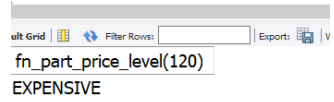
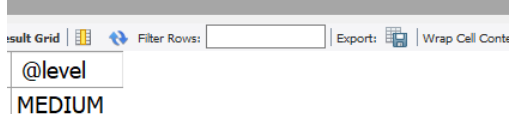
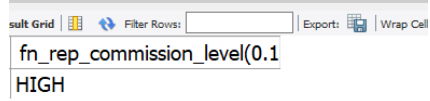
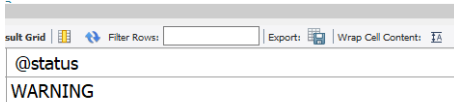
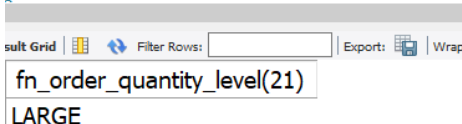
<https://www.mysqltutorial.org/mysql-stored-procedure/mysql-if-statement/>
<https://www.mysqltutorial.org/mysql-stored-procedure/mysql-case-statement/>
<https://www.mysqltutorial.org/mysql-stored-procedure/loop-in-stored-procedures/>
<https://www.mysqltutorial.org/mysql-stored-procedure/mysql-while-loop/>

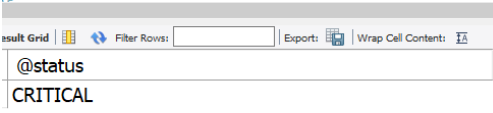
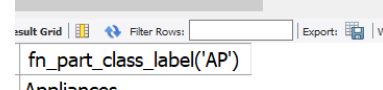
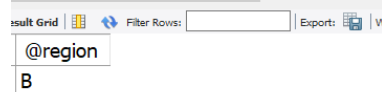
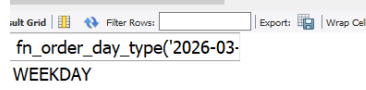
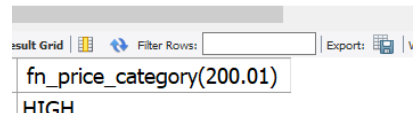
Today's Task

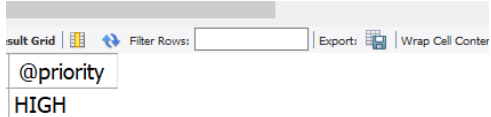
0. Database Setup:

Import the database by executing the **PremierProducts.sql** and **Data_PremierProducts.sql** files. (Same as Lab9)

Easy1 Task: IF statements	
1.	Create procedure sp_customer_credit_level. Input: IN p_customer INT Output: OUT p_level VARCHAR(20) Rules: CreditLine > 10000 → PLATINUM CreditLine 7500–10000 → GOLD CreditLine < 7500 → SILVER <pre> 48 • CALL sp_customer_credit_level(524, @level); 49 • select @level; 50 -- ----- </pre> 
2.	Create function fn_balance_status(balance DECIMAL). Return: balance < 0 → DEBT balance < 1000 → LOW otherwise → NORMAL <pre> 77 • SELECT fn_balance_status(850); </pre> 

3.	Create procedure <code>sp_part_stock_status</code>. Input: IN <code>p_part VARCHAR(10)</code> Output: OUT <code>p_status VARCHAR(20)</code> Rules: <pre> OnHand = 0 → OUT_OF_STOCK OnHand < 10 → LOW_STOCK else → AVAILABLE </pre>	<pre> 1 • CALL sp_part_stock_status('DR93', @status); 1 • select @status; </pre> 
4.	Create function <code>fn_part_price_level(price DECIMAL)</code>. Return: <pre> price < 20 → CHEAP price < 100 → NORMAL else → EXPENSIVE </pre>	<pre> 1 • SELECT fn_part_price_level(120); </pre> 
5.	Create procedure <code>sp_order_size_level</code>. Input: IN <code>p_order INT</code> Output: OUT <code>p_level VARCHAR(20)</code> Rules based on total quantity in <code>orderline</code>: <pre> < 5 → SMALL < 10 → MEDIUM else → LARGE </pre>	<pre> 1 • CALL sp_order_size_level(21617, @level); 2 • select @level; </pre> 
6.	Create function <code>fn_rep_commission_level(rate FLOAT)</code>. Return: <pre> rate < 0.05 → LOW rate < 0.1 → MEDIUM else → HIGH </pre>	<pre> 1 • SELECT fn_rep_commission_level(0.11); </pre> 
7.	Create procedure <code>sp_customer_balance_warning</code>. Rules: <pre> Balance > CreditLine → OVER_LIMIT Balance > 0.8*CreditLine → WARNING else → SAFE </pre>	<pre> 1 • CALL sp_customer_balance_warning(148, @status); 2 • select @status; </pre> 
8.	Create function <code>fn_order_quantity_level(qty INT)</code>. Return: <pre> qty < 5 → SMALL qty < 20 → MEDIUM else → LARGE </pre>	<pre> 1 • SELECT fn_order_quantity_level(21); </pre> 

9.	Create procedure sp_part_inventory_warning. Rules: OnHand < 10 → CRITICAL OnHand < 20 → LOW else → NORMAL	<pre>14 • CALL sp_part_inventory_warning('KT03', @status); 15 • SELECT @status;</pre> 
10.	Create function fn_customer_debt_ratio(balance, creditline). Return: ratio > 0.9 → RISK ratio > 0.5 → WARNING else → SAFE	<pre>8 • SELECT fn_customer_debt_ratio(3000,10000);</pre> 
Easy2 Task: CASE statements		
11.	Create procedure sp_customer_region. Output region based on Zip: 32543 → NORTH-WEST 33503 → CENTER 33363, 33321, 33146 → SOUTH else → Not in List	<pre>7 • CALL sp_customer_region(148, @region); 8 • SELECT @region;</pre> 
12.	Create function fn_part_class_label(class VARCHAR). HW → Housewares SG → Sporting Goods AP → Appliances	<pre>5 • SELECT fn_part_class_label('AP');</pre> 
13.	Create procedure sp_rep_region. Determine region based on City. Grove → A Sheldon → B Fillmore → V	<pre>5 • CALL sp_rep_region(35, @region); 6 • SELECT @region;</pre> 
14.	Create function fn_order_day_type(orderdate DATE). Weekend → WEEKEND else → WEEKDAY	<pre>7 • SELECT fn_order_day_type('2026-03-27');</pre> 
15.	Create procedure sp_customer_credit_category. CreditLine ≥ 8000 → VIP ≥ 6000 → GOLD else → NORMAL	<pre>0 • CALL sp_customer_credit_category(282, @rank); 1 • SELECT @rank;</pre> 
16.	Create function fn_price_category(price DECIMAL). < 50 → LOW < 200 → MEDIUM else → HIGH	<pre>7 • SELECT fn_price_category(200.01);</pre> 

17.	Create procedure <code>sp_order_priority</code>. Based on total order value: $< 100 \rightarrow \text{LOW}$ $< 1000 \rightarrow \text{MEDIUM}$ else $\rightarrow \text{HIGH}$	<pre>7 • CALL sp_order_priority(21613, @priority); 8 • SELECT @priority;</pre> 
	HARD Task: LOOP / WHILE / REPEAT	
18.	Create procedure <code>sp_count_customer_orders</code>. Input: <code>IN p_customer INT</code> Use <u>LOOP</u> to count the number of orders belonging to that customer. Output: <code>totalOrders</code> <pre>8 • CALL sp_count_customer_orders(608);</pre> 	
19.	Create procedure <code>sp_sum_order_quantity</code>. Input: <code>IN p_order INT</code> Use <u>WHILE</u> loop to sum NumOrdered from orderline. Output: <code>TotalQuantity</code> <pre>634 • CALL sp_sum_order_quantity(21617);</pre> 	
20.	Create function <code>fn_customer_total_spent</code>. Input: <code>p_customer INT</code> Use <u>REPEAT</u> loop to accumulate total spending from all orders: <code>SUM(NumOrdered * QuotedPrice)</code> Return: <code>TotalSpent</code> 	